



CY TECH

MASTER II MATHÉMATIQUES APPLIQUÉES

Année universitaire 2025-2026

PFE : IA Socratique

Réalisé par :

Zakaria ETTOUHAMI

Encadré par :

M. PHILIPPE VELASQUES

Table des matières

1	Chapitre 1 - l'État de l'art	3
1.1	Dialogue Socratique.	3
1.2	LLM et Prompt Engineering.	4
1.3	RAG : Retrieval Augmented Generation.	5
1.4	l'IA au service de l'éducation.	7
1.5	Problématique	8
2	Chapitre 2 - Conception et Réalisation du Système	9
2.1	Architecture globale	9
2.2	Préparation des données	10
2.2.1	Stratégie de chunking	10
2.2.2	Extraction de mots-clés avec KeyBERT	11
2.2.3	Structure du fichier JSONL	11
2.3	Indexation vectorielle	12
2.3.1	Génération des embeddings avec Sentence-BERT	12
2.3.2	Indexation avec FAISS	12
2.4	Recherche Hybride	14
2.4.1	Recherche sémantique — FAISS	14
2.4.2	Recherche par termes clés — BM25	14
2.4.3	Fusion des scores	15
2.5	Mistral AI	16
2.5.1	Choix de Mistral pour CyberSocrate	16
2.5.2	Intégration Python	17
2.6	Query Expansion	17
2.7	Multi-Retrieval	18
2.7.1	Principe	18
2.7.2	Fusion et dédoublement	18
2.7.3	Impact sur le bruit	19
2.8	BGE Reranker	20
2.8.1	Motivation	20
2.8.2	Bi-Encoder vs Cross-Encoder	20
2.8.3	Implémentation	20
2.9	Moteur Socratique	22
2.9.1	Session étudiant	22
2.9.2	Calibration hybride du niveau	22
2.9.3	Système de scoring et progression	22
2.9.4	Gestion du blocage	23
2.9.5	Prompt Engineering avancé	23
2.9.6	Conception du Prompt Socratique	24
3	Chapitre 3 - Déploiement et Interface	27
3.1	Authentification JWT	27
3.2	Base de données étudiants	27
3.3	API FastAPI	28
3.4	Tests avec Postman	28
3.5	Test de la V1 avec étudiante en cybersécurité	29

4	Chapitre 4 - Évaluation	30
4.1	Évaluation du pipeline RAG	30
4.2	Évaluation des performances	30
4.3	Évaluation humaine	30
5	Conclusion et Perspectives	31
5.1	Bilan	31
5.2	Difficultés rencontrées	31
5.3	Perspectives	32

1 Chapitre 1 - l'État de l'art

1.1 Dialogue Socratique.

Au sens premier, le dialogue socratique est un genre littéraire et philosophique de la Grèce Antique. Les dialogues socratiques sont des œuvres en prose caractérisés par la mise en scène du philosophe Socrate comme personnage principal [1]. En un sens plus restreint, les dialogues socratiques sont un sous-ensemble des dialogues rédigés par son disciple Platon où Socrate met en œuvre une méthode particulière de réfutation de son interlocuteur afin de l'aider à progresser vers la connaissance. Enfin, en un sens plus large, le dialogue socratique est une méthode d'enseignement et de psychothérapie inspirée par la méthode de réfutation mise en œuvre par Socrate dans ces dialogues.

En pédagogie [1], Le dialogue socratique désigne une méthode pédagogique d'enseignement inspiré par la réfutation mise en oeuvre par Socrate dans les dialogues de Platon dits « socratiques ». Le dialogue socratique est une stratégie qui s'appuie sur l'interrogation et dont le but est de stimuler la pensée critique de l'apprenant, en l'amenant à prendre conscience de ce qu'il sait implicitement, puis de l'exprimer et de le juger. Cette stratégie situe l'apprenant au centre de son apprentissage.

TABLE 1 – Les principales approches pédagogiques

Approche	Définition
Socratique	Basée sur le questionnement guidé, elle amène l'apprenant à construire lui-même ses connaissances à travers une série de questions progressives.
Transmissive	L'enseignant transmet directement le savoir (cours magistral). L'apprenant joue un rôle passif de récepteur.
Constructiviste	L'apprenant construit ses connaissances par lui-même à partir de ses expériences et de ses interactions avec son environnement.
Socioconstructiviste	Extension du constructivisme mettant l'accent sur l'apprentissage par interaction avec les autres (travail collaboratif, échanges entre pairs).
Par compétences	Centrée sur le développement de compétences pratiques (savoir-faire, savoir-être) plutôt que sur la simple acquisition de connaissances.
Par problème (PBL)	Les apprenants résolvent des problèmes concrets pour apprendre, ce qui favorise la réflexion critique et l'autonomie.

1.2 LLM et Prompt Engineering.

LLM (Large Language Model) est un modèle de langage possédant un grand nombre de paramètres (généralement plus d'un milliard). Ce sont des réseaux de neurones profonds entraînés sur de grandes quantités de texte non étiqueté utilisant l'apprentissage auto-supervisé [2]. Les LLM sont apparus vers 2017 et ont été utilisés pour la mise en œuvre d'agents conversationnels. Ils constituent également le socle technique des moteurs de recherche génératifs, qui synthétisent directement une réponse en langage naturel à partir de multiples sources plutôt que de retourner une liste de liens. et ont été utilisés pour la mise en œuvre d'agents conversationnel et ont été utilisés pour automatiser de nombreuses tâches métier.

Fonctionnalités de LLM les plus couramment utilisées par les entreprises ils permettent aux utilisateurs de générer des phrases et des paragraphes cohérents et adaptés au contexte en réponse à un prompt, les LLM peuvent automatiser de nombreuses tâches métier : Synthèse de texte ,Traduction ,Génération de texte et d'image, Écriture et débogage de code ,Recherche sur le Web ,Service client et analyse du ressenti des clients, Assistants virtuels/chatbots ,Classification de texte/document ,Vérification et approbation automatisées de documents ,Réponses de base de connaissances et Rédaction éditoriale et technique [3]

Les LLM présentent également certaines limites :

Comportement non déterministe : les LLM peuvent produire des réponses variables et parfois même contradictoires à la même requête, car leurs sorties dépendent de modèles probabilistes plutôt que de règles fixes.

Hallucinations : ces modèles peuvent parfois générer des informations incorrectes ou absurdes, en s'appuyant sur des schémas appris plutôt que sur la précision factuelle.

Injections de requêtes : les LLM peuvent être vulnérables aux attaques par injection de requêtes, où les utilisateurs créent des requêtes d'entrée qui manipulent le modèle pour qu'il s'écarte de sa fonction prévue ou produise des résultats indésirables.

Par conséquent, les utilisateurs doivent vérifier les résultats générés par les LLM avant de prendre des mesures conséquentes.

Prompt Engineering est le processus de conception et de structuration d'un prompt, de sorte qu'il soit efficacement interprété et compris par un modèle d'intelligence artificielle générative. Un prompt (aussi appelé requête) est une description textuelle de la tâche qu'une IA doit effectuer.

Un prompt, pour un modèle de langage produisant du texte, peut être une requête telle que « qu'est-ce que le petit théorème de Fermat ? ». Ou bien une commande comme « écrire un poème sur la chute des feuilles », ou une déclaration plus longue incluant du contexte, des instructions, et l'historique des conversations. L'ingénierie de prompt peut impliquer de formuler une requête, de spécifier un style, de fournir un contexte pertinent, ou d'attribuer un rôle à l'IA.

1.3 RAG : Retrieval Augmented Generation.

RAG est une technique d'optimisation de réponses de modèle de langage en intelligence artificielle générative (IA). Cette méthode permet notamment d'améliorer la qualité de réponses aux requêtes en permettant aux grands modèles de langage d'exploiter des ressources de données supplémentaires sans ré-entraînement[4].

Les cas d'utilisation incluent l'accès via un chatbot aux données internes de l'entreprise ou la diffusion d'informations vérifiées provenant exclusivement de sources fiables et reconnues. Avec la RAG, les LLM ne répondent pas aux requêtes des utilisateurs tant qu'ils n'ont pas consulté un ensemble spécifique de documents. Ces documents complètent les informations provenant des données d'entraînement préexistantes du LLM. Cela permet aux LLM d'utiliser des informations spécifiques à un domaine qui ne sont pas disponibles dans les données d'entraînement.

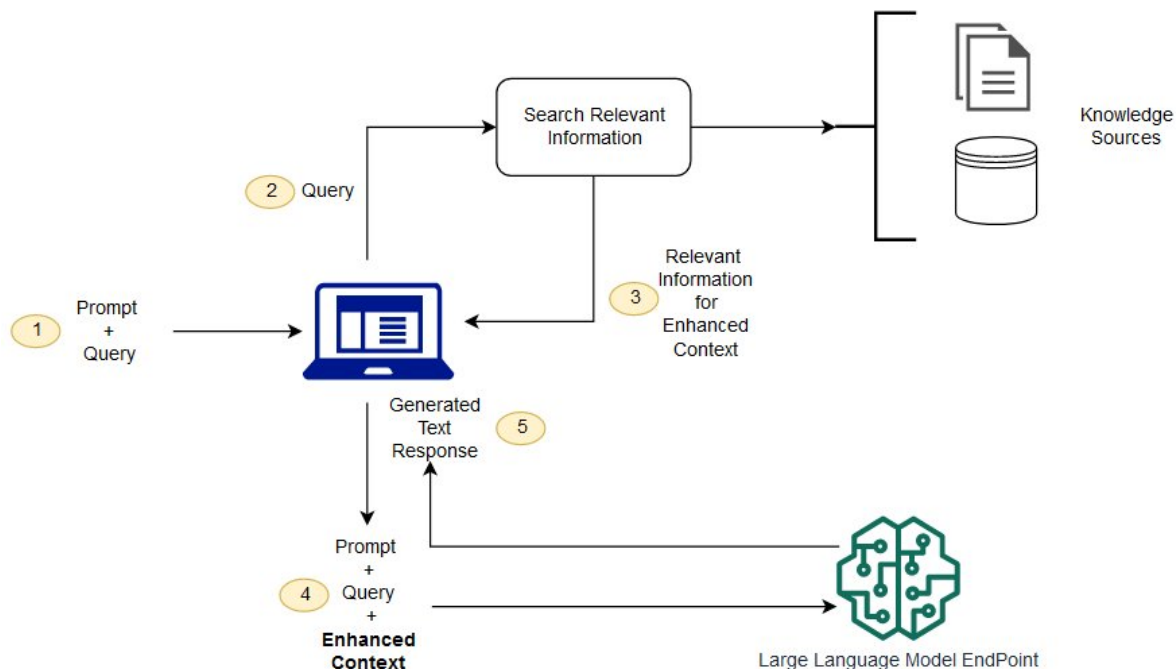


FIGURE 1 – Architecture globale du RAG

Le RAG réduit également la nécessité de réentraîner les LLM avec de nouvelles données [4], ce qui permet de réaliser des économies de ressources informatiques et financières. Au-delà des gains d'efficacité, le RAG permet également aux LLM d'inclure des sources dans leurs réponses, afin que les utilisateurs puissent vérifier les sources citées. Cela offre une plus grande transparence, car les utilisateurs peuvent recouper les contenus récupérés afin de s'assurer de leur exactitude et de leur pertinence. Le terme RAG a été introduit pour la première fois dans un article de recherche publié en 2020.

Quick definition: What is RAG?

Without RAG: LLM model has to be the only source of knowledge

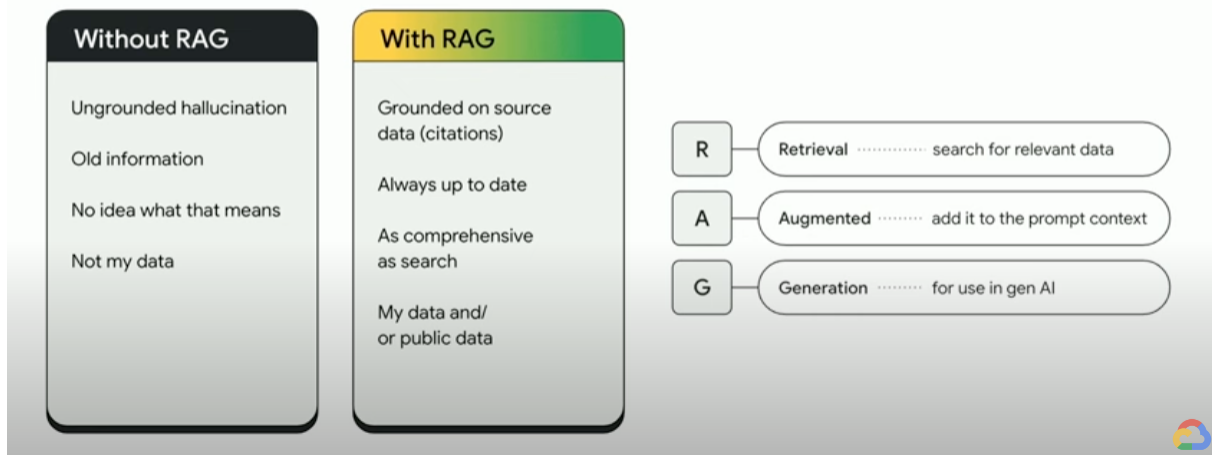


FIGURE 2 – Illustration du concept RAG et de ses avantages

Les LLM et le RAG présentent quand même **certaines limites** comme fournir des informations incorrectes. Par exemple, lorsque Google a présenté pour la première fois son chatbot Google Bard (rebaptisé par la suite Gemini) [4], le LLM a fourni des informations incorrectes sur le télescope spatial James Webb. Cette erreur a contribué à une baisse de 100 milliards de dollars de la valeur boursière de l'entreprise. Le RAG est utilisé pour éviter ces erreurs, mais il ne résout pas tous les problèmes. Par exemple, les LLM peuvent générer des informations erronées même lorsqu'ils s'appuient sur des sources factuellement correctes s'ils interprètent mal le contexte. La MIT Technology Review donne l'exemple d'une réponse générée par l'IA affirmant que «les États-Unis ont eu un président musulman, Barack Hussein Obama». Le modèle a extrait cette information d'un ouvrage universitaire intitulé de manière rhétorique «Barack Hussein Obama : le premier président musulman des États-Unis ? ». Le LLM ne «connaissait» ni ne «comprenait» le contexte du titre, générant ainsi une réponse fausse.

Un système RAG peut récupérer des sources correctes mais trompeuses, ce qui peut conduire à des erreurs d'interprétation. Le modèle peut, par exemple, extraire une information hors contexte ou avoir des difficultés à gérer des sources contradictoires. Dans certains cas, il peut même combiner des données provenant de différentes sources, produisant ainsi une réponse incohérente ou trompeuse. Selon MIT Technology Review, ces limites sont dues au fait que les systèmes RAG ne comprennent pas réellement les informations qu'ils exploitent.

1.4 l'IA au service de l'éducation.

Aujourd'hui, force est de constater que les IA "grand public" sont déjà largement utilisées en éducation par :

->**les étudiants** : si certains les utilisent pour faire leurs devoirs à leur place, d'autres s'en saisissent efficacement pour réviser, s'entraîner, approfondir leurs connaissances ;

->**les enseignants** : certains s'en emparent pour préparer leurs cours, concevoir des évaluations, adapter les contenus aux besoins particuliers des élèves, voire les aider dans la correction et le retour personnalisé ;

Aujourd'hui, il existe plein d'IA éducatives différentes :

TABLE 2 – Outils d'IA appliqués à l'éducation

Catégorie	Outil	Description	Limites
Tuteurs IA	Khanmigo	Tuteur intelligent lié à Khan Academy, explique les cours et pose des questions adaptatives.	Réponses directes, pas d'approche socratique.
	Duolingo Max	Apprentissage des langues avec IA générative, conversations simulées.	Limité aux langues, pas adapté aux matières techniques.
	GoStudent Learning	Accompagnement personnalisé en temps réel adapté au niveau de l'apprenant.	Contenu générique, non ancré dans un cours spécifique.
Outils de révision	NotebookLM	Résume des documents et génère des guides d'étude personnalisés.	Pas interactif, ne guide pas l'apprenant.
	Quizlet	Flashcards intelligentes avec répétition espacée.	QCM uniquement, pas de raisonnement guidé.
	MyMap AI	Crée des fiches et cartes mentales à partir de cours.	Passif, l'étudiant ne construit pas ses connaissances.

Les plus utilisés par la masse reste quand même les 3 IA géante : ChatGPT (OpenAI), Gemini (Google), Claude (Anthropic)

1.5 Problématique

Les outils d'intelligence artificielle éducatifs actuels, qu'il s'agisse de Khanmigo, ChatGPT ou encore NotebookLM, **partagent une limite fondamentale** : ils fournissent des réponses directes et immédiates aux questions des apprenants. Si cette approche permet un accès rapide à l'information, elle ne favorise pas le développement du raisonnement critique ni la construction autonome des connaissances.

En effet, un étudiant qui obtient immédiatement la réponse à sa question mémorise sans comprendre. Il ne développe pas la capacité d'analyse et de déduction indispensable dans des domaines techniques complexes comme la cybersécurité, où chaque incident exige une réflexion méthodique et rigoureuse.

"Tell me and I forget, teach me and I may remember, involve me and I learn."

— Benjamin Franklin

C'est dans ce contexte que s'inscrit notre projet. Nous proposons **CyberSocrate**, un assistant pédagogique socratique destiné aux étudiants ingénieurs de troisième année (ING3) en cybersécurité à CY Tech. Contrairement aux assistants existants, CyberSocrate ne fournit jamais la réponse directement. Il guide l'étudiant à travers une série de questions progressives et adaptatives, l'amenant à construire lui-même sa compréhension des concepts fondamentaux de la cybersécurité.

Cette approche, inspirée de la méthode socratique, vise à :

- Stimuler le raisonnement analytique de l'étudiant
- Adapter le niveau de questionnement au profil de l'apprenant
- Ancrer les connaissances dans le contenu des cours ING3 CY Tech
- Favoriser une progression mesurable et gamifiée

Ainsi, **la question centrale de ce projet** est la suivante : *Comment concevoir un assistant pédagogique intelligent capable de guider les étudiants en cybersécurité par le questionnement socratique, tout en s'adaptant dynamiquement à leur niveau de compréhension ?*

2 Chapitre 2 - Conception et Réalisation du Système

2.1 Architecture globale

L'architecture de CyberSocrate s'inspire des moteurs de recherche modernes enrichis par l'intelligence artificielle. Elle repose sur le paradigme **RAG (Retrieval-Augmented Generation)**, qui combine une source de données interne avec un modèle de langage génératif pour produire des réponses contextualisées et pédagogiquement guidées.

Contrairement aux assistants classiques qui s'appuient uniquement sur les connaissances générales du LLM, CyberSocrate ancre chaque interaction dans le contenu des cours dispensés, garantissant ainsi la cohérence pédagogique avec le programme ING3.

L'architecture globale du système se décompose en quatre couches principales :

- **Couche données** : corpus de documents pédagogiques découpés en chunks et indexés sous forme vectorielle et lexicale
- **Couche retrieval** : pipeline de recherche hybride combinant FAISS, BM25, Query Expansion et BGE Reranker pour récupérer les passages les plus pertinents
- **Couche génération** : moteur socratique basé sur Mistral AI qui transforme les chunks récupérés en questions guidées adaptées au niveau de l'étudiant
- **Couche application** : API REST FastAPI sécurisée par JWT et interface React exposant les fonctionnalités aux étudiants et au professeur

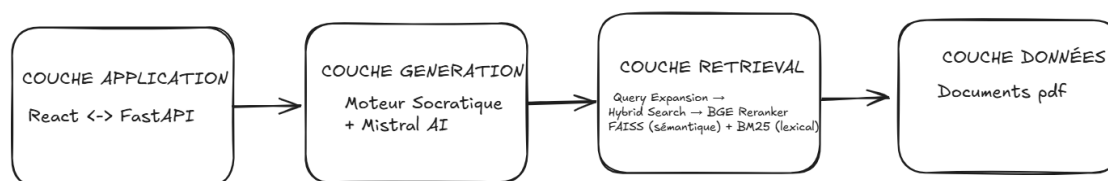


FIGURE 3 – Architecture globale de CyberSocrate

Les sections suivantes détaillent chacune des composantes de cette architecture, depuis la **préparation des données** jusqu'au **moteur socratique**.

2.2 Préparation des données

La construction de la base de connaissances de CyberSocrate repose sur des documents pédagogiques utilisés dans le cadre du cours ING3 Cybersécurité à CY Tech ou des mémoires d'universités trouvés sur Internet, ceci fut une des grosses difficultés rencontrés dans ce projet, la collecte de données pertinent à chunk sur des thématiques de Cybersécurité.

Ces documents ont été extraits au format texte à l'aide de la bibliothèque `pypdf`. Certains fichiers étant protégés par chiffrement, un déchiffrement préalable avec mot de passe vide a été appliqué :

Listing 1 – Extraction du texte PDF

```
1 reader = PdfReader("document.pdf")
2 reader.decrypt("")
3 text = reader.pages[i].extract_text()
```

Un nettoyage du texte extrait a ensuite été effectué afin de supprimer les éléments non pertinents tels que les en-têtes répétitifs, les numéros de page, les ligatures (`fi` → `fi`) et les apostrophes typographiques.

2.2.1 Stratégie de chunking

Le texte nettoyé est découpé en chunks (segments) de taille fixe avec chevauchement (*overlap*) afin de préserver le contexte entre les segments adjacents. La stratégie adoptée est la suivante :

- Taille du chunk : **300 mots**
- Chevauchement : **30 mots**

Le nombre de chunks N produits à partir d'un texte de W mots est donné par la formule :

$$N = \left\lceil \frac{W - \text{overlap}}{\text{taille} - \text{overlap}} \right\rceil = \left\lceil \frac{W - 30}{300 - 30} \right\rceil = \left\lceil \frac{W - 30}{270} \right\rceil \quad (1)$$

Le chevauchement de 30 mots garantit qu'aucune information importante ne soit perdue à la frontière entre deux chunks consécutifs. Voici la fonction essentiel du chunking qu'a été utilisé sur tout les documents qu'ont alimenté notre base de connaissance or chacune avec taille et overlap différent (pas forcément 300) :

Listing 2 – Fonction de chunking avec chevauchement

```
1 def chunker(texte, taille=300, overlap=30):
2     mots = texte.split()
3     chunks = []
4     i = 0
5
6     while i < len(mots):
7         chunk = mots[i:i + taille]
8         chunks.append(" ".join(chunk))
9         i += taille - overlap
10
11     return chunks
```

2.2.2 Extraction de mots-clés avec KeyBERT

Afin d'enrichir chaque chunk avec des termes techniques représentatifs, nous avons utilisé **KeyBERT**, une bibliothèque d'extraction de mots-clés basée sur les embeddings BERT. Les paramètres retenus sont les suivants [5] :

```
1 keywords = kw_model.extract_keywords(  
2     chunk,  
3     keyphrase_ngram_range=(1, 2),    # unigrammes et bigrammes  
4     use_mmr=True,                    # diversité maximale  
5     diversity=0.7,                  # éviter la redondance  
6     top_n=5                          # 5 keywords par chunk  
7 )
```

L'algorithme **MMR** (Maximal Marginal Relevance) permet d'obtenir des keywords diversifiés en évitant la redondance sémantique. Un filtre supplémentaire est appliqué pour ne conserver que les termes pertinents dans le domaine cybersécurité :

$$kw \in \text{Keywords} \iff |kw| > 4 \vee kw \in \text{TERMES_CYBER_COURTS} \quad (2)$$

L'utilisation de KeyBERT s'explique par la volonté de combiner **deux types de recherche dans notre pipeline : une recherche sémantique par embeddings vectoriels et une recherche par termes clés exacts via BM25**. Les keywords extraits enrichissent ainsi l'index BM25 avec les termes techniques propres à chaque chunk.

2.2.3 Structure du fichier JSONL

L'ensemble des chunks est stocké dans un fichier au format **JSONL** (JSON Lines), où chaque ligne représente un chunk avec ses métadonnées associées :

```
1 {  
2     "id": "FOR-001",  
3     "source": "Forensics.pdf",  
4     "chunk_index": 0,  
5     "domain": "forensics",  
6     "content": "La forensique numerique consiste a...",  
7     "keywords": ["investigation numerique", "artefacts",  
8                 "chaine de preuve"],  
9     "needs_manual": true,  
10    "related_ids": []  
11 }
```

Le champ **domain** permet le filtrage par thématique lors de la recherche hybride, tandis que **keywords** enrichit l'index BM25 avec des termes techniques représentatifs du contenu.

2.3 Indexation vectorielle

2.3.1 Génération des embeddings avec Sentence-BERT

Chaque chunk est transformé en vecteur dense de dimension 384 à l'aide du modèle **Sentence-BERT**, plus précisément le modèle multilingue **paraphrase-multilingual-MiniLM-L12-v2**. Ce modèle a été retenu car il supporte nativement le français et l'anglais [6], ce qui correspond à la nature mixte de notre corpus et c'est aussi un modèle avec lequel je suis très familier vu que je comprends parfaitement comment il fonctionne (vu en cours de NLP + stage).

Listing 3 – Génération des embeddings

```
1 from sentence_transformers import SentenceTransformer
2
3 modele = SentenceTransformer(
4     "paraphrase-multilingual-MiniLM-L12-v2"
5 )
6 embeddings = modele.encode(
7     contenus,
8     batch_size=32,
9     convert_to_numpy=True
10 )
11 # embeddings.shape = (164, 384)
```

Chacun des chunks est ainsi représenté par un vecteur $\mathbf{v} \in \mathbb{R}^{384}$, capturant sa sémantique dans un espace vectoriel de haute dimension.

2.3.2 Indexation avec FAISS

Les vecteurs générés sont indexés à l'aide de la bibliothèque **FAISS** (Facebook AI Similarity Search), qui permet une recherche efficace des voisins les plus proches dans un espace vectoriel. Nous avons évalué trois méthodes d'indexation différentes [7] :

IndexFlatIP — Produit scalaire (similarité cosinus) Cette méthode calcule le produit scalaire entre deux vecteurs normalisés, ce qui est équivalent à la similarité cosinus :

$$\text{score}(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^d u_i \cdot v_i \quad (3)$$

Après normalisation L2 des vecteurs ($\|\mathbf{v}\|_2 = 1$), cette formule est équivalente à :

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \cdot \|\mathbf{v}\|_2} \quad (4)$$

IndexFlatL2 — Distance euclidienne Cette méthode mesure la distance géométrique entre deux vecteurs :

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^d (u_i - v_i)^2} \quad (5)$$

IndexIVFFlat — Clustering k-means Cette méthode divise l’espace vectoriel en k clusters via k-means, puis limite la recherche aux clusters les plus proches :

$$C^* = \arg \min_{C_j} d(\mathbf{q}, \boldsymbol{\mu}_j), \quad j = 1, \dots, k \quad (6)$$

où $\mathbf{q}$ est le vecteur de la question et $\boldsymbol{\mu}_j$ est le centroïde du cluster C_j .

Choix retenu Le tableau 3 compare les trois méthodes selon les critères pertinents pour notre cas d’usage :

TABLE 3 – Comparaison des méthodes d’indexation FAISS

Critère	IndexFlatIP	IndexFlatL2	IndexIVFFlat
Précision	Exacte	Exacte	Approximative
Métrique	Cosinus	Euclidienne	Euclidienne
Vitesse	Moyenne	Moyenne	Rapide
Corpus idéal	< 100K	< 100K	> 1M
Adapté sentence- transformers	Oui	Non	Partiel

Nous avons retenu **IndexFlatIP** car la similarité cosinus correspond à la métrique d’entraînement native de Sentence-BERT, garantissant des résultats optimaux, mais peut-être si on élargit dans le futur ce projet avec + de données ça serait très intéressant de passer à **IndexIVFFlat**.

2.4 Recherche Hybride

La recherche hybride constitue l’une des composantes les plus importantes de notre pipeline RAG. Elle combine deux approches complémentaires afin de maximiser la pertinence des chunks retournés : une recherche sémantique dense via FAISS et une recherche par termes exacts via BM25.

2.4.1 Recherche sémantique — FAISS

La question de l’étudiant est encodée en vecteur dense à l’aide du même modèle Sentence-BERT utilisé lors de l’indexation, puis normalisée avant la recherche :

Listing 4 – Recherche vectorielle FAISS

```
1 vecteur_question = modele.encode(  
2     [question], convert_to_numpy=True  
3 )  
4 faiss.normalize_L2(vecteur_question)  
5 scores_faiss, indices_faiss = index_faiss.search(  
6     vecteur_question, top_k  
7 )
```

FAISS retourne les k chunks dont les vecteurs sont les plus proches sémantiquement de la question.

2.4.2 Recherche par termes clés — BM25

En parallèle, l’algorithme **BM25** (Best Match 25) calcule un score de pertinence basé sur la fréquence des termes. La formule de scoring BM25 pour un document d par rapport à une requête q est :

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (7)$$

où :

- $f(t, d)$: fréquence du terme t dans le document d
- $|d|$: longueur du document d
- avgdl : longueur moyenne des documents du corpus
- $k_1 = 1.5$ et $b = 0.75$: paramètres de calibration
- $\text{IDF}(t) = \log \frac{N - n(t) + 0.5}{n(t) + 0.5}$: fréquence inverse de document

L’index BM25 est construit sur le contenu des chunks enrichi des keywords KeyBERT, afin de renforcer le poids des termes techniques :

Listing 5 – Construction de l’index BM25

```
1 corpus_tokenise = [  
2     chunk["content"].lower().split() + chunk["keywords"]  
3     for chunk in chunks  
4 ]  
5 bm25 = BM25Okapi(corpus_tokenise)
```

2.4.3 Fusion des scores

Les scores FAISS et BM25 opèrent sur des échelles différentes. Une normalisation **Min-Max** est appliquée sur chacun afin de les ramener dans l'intervalle $[0, 1]$:

$$\hat{s} = \frac{s - s_{\min}}{s_{\max} - s_{\min}} \quad (8)$$

Le score hybride final est ensuite calculé par combinaison linéaire pondérée par un paramètre $\alpha \in [0, 1]$:

$$s_{\text{hybride}} = \alpha \cdot \hat{s}_{\text{FAISS}} + (1 - \alpha) \cdot \hat{s}_{\text{BM25}} \quad (9)$$

Avec $\alpha = 0.5$, les deux modalités contribuent de manière équilibrée au score final. Les chunks sont ensuite triés par score décroissant et les k meilleurs sont retenus.

Listing 6 – Implémentation de la recherche hybride

```
1 def recherche_hybride(question, chunks, index_faiss,
2                       bm25, modele, top_k=10, alpha=0.5):
3
4     # 1 - Vector Search FAISS
5     vecteur_question = modele.encode(
6         [question], convert_to_numpy=True
7     )
8     faiss.normalize_L2(vecteur_question)
9     scores_faiss, indices_faiss = index_faiss.search(
10        vecteur_question, top_k
11    )
12    scores_faiss = scores_faiss[0]
13    indices_faiss = indices_faiss[0]
14
15    # Normalisation Min-Max
16    if scores_faiss.max() > scores_faiss.min():
17        scores_faiss = (scores_faiss - scores_faiss.min()) \
18            / (scores_faiss.max() - scores_faiss.min())
19
20    # 2 - BM25 Search
21    scores_bm25 = bm25.get_scores(question.lower().split())
22    if scores_bm25.max() > scores_bm25.min():
23        scores_bm25 = (scores_bm25 - scores_bm25.min()) \
24            / (scores_bm25.max() - scores_bm25.min())
25
26    # 3 - Fusion hybride
27    scores_hybrides = np.zeros(len(chunks))
28    for i, idx in enumerate(indices_faiss):
29        scores_hybrides[idx] += alpha * scores_faiss[i]
30    scores_hybrides += (1 - alpha) * scores_bm25
31
32    # 4 - Top-K
33    top_indices = np.argsort(scores_hybrides)[::-1][:top_k]
34    return [{"chunk": chunks[i],
35            "score_hybride": float(scores_hybrides[i])}
36            for i in top_indices]
```

2.5 Mistral AI

Mistral AI est une entreprise française fondée en 2023, spécialisée dans le développement de modèles de langage open-source performants et efficaces. Ses modèles se distinguent par leur rapport **qualité/performance** élevé, rivalisant avec les grands modèles propriétaires tout en restant accessibles via une API



FIGURE 4 – Mistral AI

Mistral AI propose plusieurs modèles adaptés à différents cas d’usage :

TABLE 4 – Principaux modèles Mistral AI

Modèle	Description
<code>mistral-large-latest</code>	Modèle le plus puissant, adapté aux tâches complexes de raisonnement.
<code>mistral-small-latest</code>	Modèle équilibré entre performance et coût, idéal pour les applications temps réel.
<code>mistral-embed</code>	Modèle dédié à la génération d’embeddings textuels.
<code>codestral</code>	Modèle spécialisé dans la génération de code.

2.5.1 Choix de Mistral pour CyberSocrate

Dans le cadre de notre projet, nous utilisons le modèle `mistral-small-latest` pour deux fonctions distinctes du pipeline :

- **Query Expansion** : reformulation de la question de l’étudiant en plusieurs variantes techniques
- **Moteur Socratique** : génération des questions guidées adaptées au niveau et au score de l’étudiant

2.5.2 Intégration Python

L'accès à l'API Mistral est réalisé via la bibliothèque officielle `mistralai` :

Listing 7 – Initialisation du client Mistral

```
1 from mistralai import Mistral
2
3 mistral_client = Mistral(api_key="MISTRAL_API_KEY")
```

2.6 Query Expansion

Lorsqu'un étudiant pose une question, il arrive fréquemment que sa formulation ne corresponde pas exactement aux termes utilisés dans les chunks de la base de connaissances. La **Query Expansion** consiste à enrichir la question initiale en générant plusieurs reformulations sémantiquement équivalentes via Mistral, afin d'améliorer le recall de la recherche hybride.

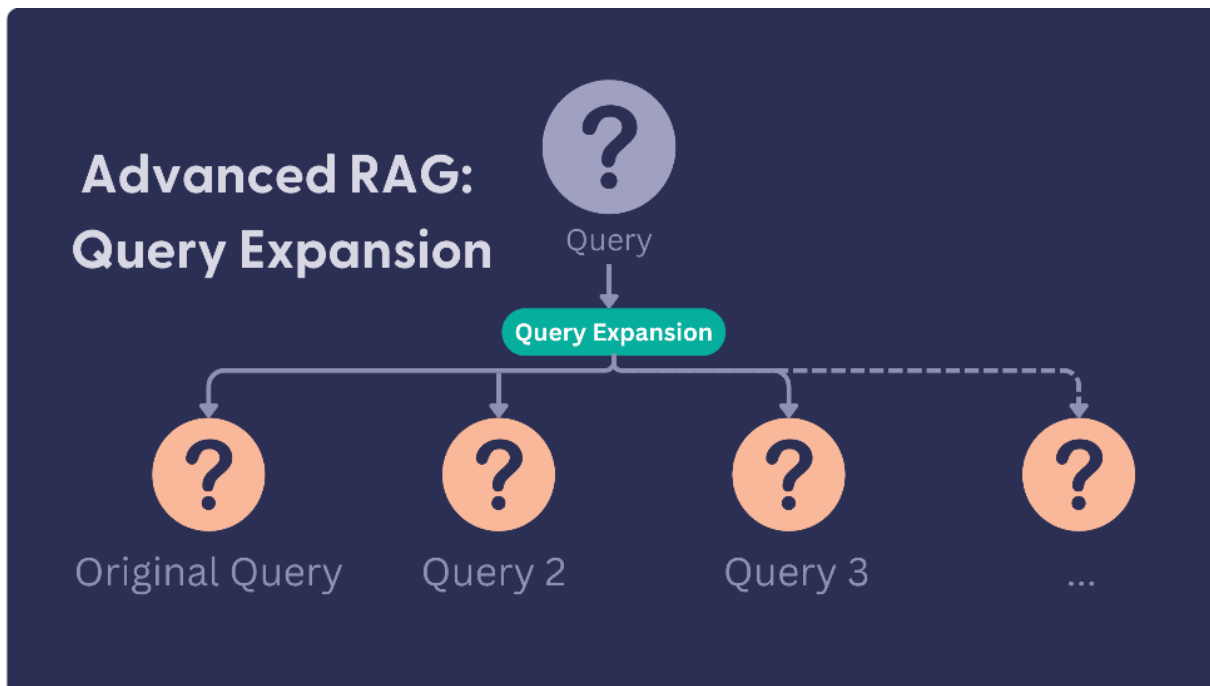


FIGURE 5 – Une query expansion

Le flow est le suivant :

1. L'étudiant pose une question q_0
2. Mistral génère n reformulations $\{q_1, q_2, q_3\}$
3. La recherche hybride est effectuée sur chacune des $n + 1$ variantes

$$Q = \{q_0, q_1, q_2, q_3\} \xrightarrow{\text{Hybrid Search}} \bigcup_{q_i \in Q} \text{top-k}(q_i) \quad (10)$$

2.7 Multi-Retrieval

2.7.1 Principe

Le **Multi-Retrieval** est une technique qui consiste à effectuer plusieurs recherches hybrides en parallèle, une pour chaque variante générée par la Query Expansion [9]. Contrairement à une recherche simple sur une seule question, le Multi-Retrieval exploite l'ensemble des reformulations $Q = \{q_0, q_1, q_2, q_3\}$ pour maximiser les chances de retrouver les chunks les plus pertinents.

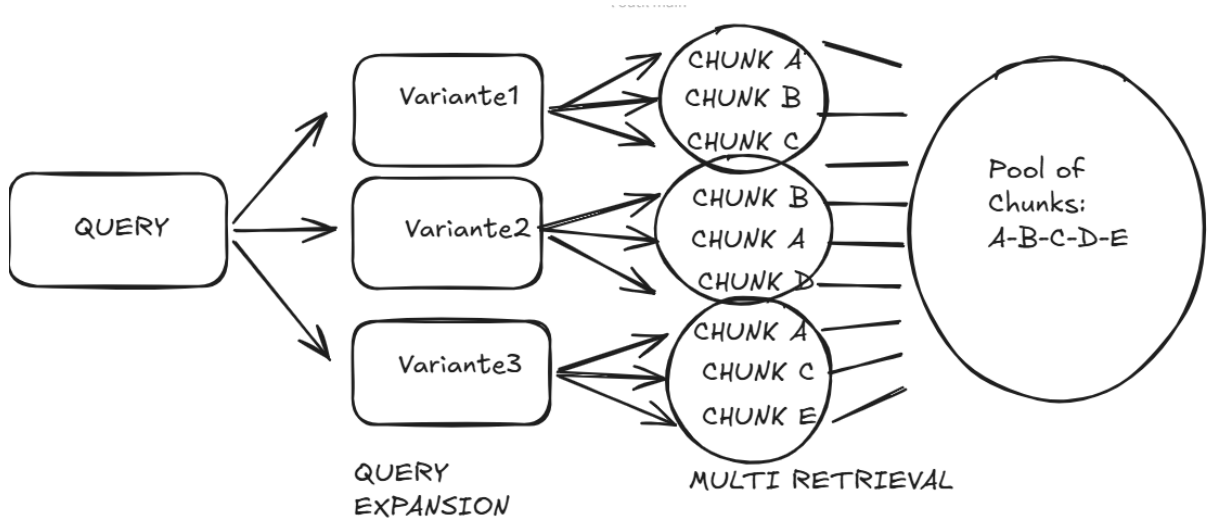


FIGURE 6 – Query expansion + Multi Retrieval via recherche Hybride

2.7.2 Fusion et dédoublonnage

Chaque recherche hybride retourne un ensemble de chunks avec leurs scores associés. Les résultats des $n + 1$ recherches sont ensuite fusionnés en un seul ensemble dédoublonné, en conservant le meilleur score pour chaque chunk :

$$s_{\text{final}}(c) = \max_{q_i \in Q} s_{\text{hybride}}(c, q_i) \quad (11)$$

Listing 8 – Fusion des résultats Multi-Retrieval

```
1 tous_les_resultats = {}
2 for q in questions: # q0, q1, q2, q3
3     resultats = recherche_hybride(q, chunks,
4                                   index_faiss, bm25, modele)
5     for r in resultats:
6         chunk_id = r["chunk"]["id"]
7         if chunk_id not in tous_les_resultats:
8             tous_les_resultats[chunk_id] = r
9         elif r["score_hybride"] > \
10             tous_les_resultats[chunk_id]["score_hybride"]:
11             tous_les_resultats[chunk_id] = r
12
13 resultats_fusionnes = list(tous_les_resultats.values())
```

2.7.3 Impact sur le bruit

La combinaison de la Query Expansion et du Multi-Retrieval améliore significativement le recall du système — les chunks pertinents ont plus de chances d’être retrouvés grâce aux multiples reformulations. Cependant, cette approche introduit inévitablement du **bruit** dans les résultats :

- Les variantes mal reformulées par Mistral peuvent ramener des chunks hors sujet
- Le nombre de chunks candidats peut atteindre ~ 20 chunks après fusion
- Certains chunks ont un score artificiellement élevé sur une variante non pertinente

Ce bruit est pris en charge en aval par le **BGE Reranker** (section 2.8), qui analyse la relation directe entre la question originale q_0 et chaque chunk retourné, réduisant ainsi les ~ 20 chunks bruités à **3 chunks finaux** réellement pertinents.

2.8 BGE Reranker

2.8.1 Motivation

Après la **Query Expansion** et le **Multi-Retrieval**, le pool de chunks candidats peut contenir une vingtaine de résultats dont certains sont bruités ou peu pertinents. La recherche hybride FAISS + BM25 évalue la pertinence de chaque chunk **indépendamment** de la question, elle mesure une proximité sémantique ou lexicale globale, sans analyser la relation directe entre la question et le chunk.

Le **BGE Reranker** résout ce problème en analysant chaque paire (question originale, chunk) **conjointement**, produisant un score de pertinence beaucoup plus précis.

2.8.2 Bi-Encoder vs Cross-Encoder

Il existe deux architectures principales pour le reranking :

TABLE 5 – Comparaison Bi-Encoder et Cross-Encoder

Critère	Bi-Encoder	Cross-Encoder
Fonctionnement	Encode question et chunk séparément	Encode la paire (question, chunk) ensemble
Score	Similarité cosinus entre deux vecteurs	Score de pertinence direct
Précision	Moyenne	Élevée
Vitesse	Rapide	Plus lente
Usage	Retrieval (FAISS)	Reranking

Le **Cross-Encoder** est plus précis car il capture les interactions fines entre les tokens de la question et ceux du chunk via le mécanisme d'attention de l'architecture Transformer :

$$s_{\text{BGE}}(q, c) = \text{CrossEncoder}([q; c]) \quad (12)$$

où $[q; c]$ désigne la concaténation de la question q et du chunk c passés ensemble dans le modèle.

2.8.3 Implémentation

Listing 9 – Fonction de reranking BGE

```
1 def reranker_chunks(question, resultats, reranker, top_k=3):
2     # Construire les paires (question, chunk)
3     paires = [
4         [question, r["chunk"]["content"]]
5         for r in resultats
6     ]
7
8     # Calculer les scores BGE
```

```

9     scores_bge = reranker.predict(paires)
10
11     # Ajouter le score BGE a chaque resultat
12     for i, r in enumerate(resultats):
13         r["score_bge"] = float(scores_bge[i])
14
15     # Trier par score BGE decroissant
16     resultats_rerankes = sorted(
17         resultats,
18         key=lambda x: x["score_bge"],
19         reverse=True
20     )
21
22     return resultats_rerankes[:top_k]

```

Le reranker prend en entrée les ~ 20 chunks issus du Multi-Retrieval et retourne les **3 chunks** les plus pertinents par rapport à la question originale q_0 , éliminant ainsi le bruit introduit par la Query Expansion et le Multi-Retrieval.

Il élimine aussi le bruit en effet l'architecture combinant **Query Expansion**, **Multi-Retrieval** et **BGE Reranker** constitue un pipeline optimal pour minimiser les hallucinations du LLM et améliorer la performance du RAG. En effet, les hallucinations dans les systèmes RAG surviennent principalement lorsque le contexte fourni au LLM est peu pertinent ou bruité.

2.9 Moteur Socratique

Le moteur socratique constitue le cœur de CyberSocrate. C'est lui qui transforme les chunks récupérés par le pipeline RAG en questions guidées adaptées au niveau et à la progression de l'étudiant. Contrairement aux assistants classiques qui fournissent une réponse directe, le moteur socratique ne donne jamais la réponse => il guide l'étudiant vers elle par le questionnement.

2.9.1 Session étudiant

Chaque interaction est maintenue dans une **session** qui conserve l'état de l'étudiant tout au long de la conversation :

Listing 10 – Structure de la session étudiant

```
1 session = {
2     "niveau_declare"      : "debutant",  # declare par l'etudiant
3     "niveau_estime"      : "debutant",  # detecte par le LLM
4     "niveau_final"       : "debutant",  # utilise pour le prompt
5     "score"              : 0,           # 0 a 3
6     "nb_echanges"        : 0,           # nombre d'echanges
7     "calibration_terminee": False,      # True apres 3 echanges
8     "historique"         : [],          # historique des echanges
9     "bloque"             : 0            # compteur blocage
10 }
```

2.9.2 Calibration hybride du niveau

L'un des aspects les plus importants du moteur socratique est la détection automatique du niveau de l'étudiant. Nous adoptons une **approche hybride** combinant deux mécanismes complémentaires :

- **Niveau déclaratif** : l'étudiant déclare son niveau au démarrage (débutant, intermédiaire, avancé). Ce niveau sert de point de départ.
- **Niveau implicite** : le LLM analyse le vocabulaire et la précision des réponses de l'étudiant pour ajuster dynamiquement le niveau estimé.

$$\text{niveau_final} = \begin{cases} \text{niveau_declare} & \text{si calibration non terminée} \\ \text{niveau_estime} & \text{si calibration terminée (3 échanges)} \end{cases} \quad (13)$$

Cette calibration s'effectue sur les 3 premiers échanges — si l'étudiant utilise un vocabulaire technique précis, le niveau est automatiquement rehaussé, et inversement.

2.9.3 Système de scoring et progression

Le moteur socratique intègre un système de scoring qui mesure la progression de l'étudiant sur chaque question :

TABLE 6 – Style de questionnement selon le score

Score	Style	Exemple
0/3	Questions fondamentales sur les concepts cyber	"Quelle différence entre une trace volatile et persistante?"
1/3	Questions sur le "pourquoi" technique	"Pourquoi ce registre persiste après suppression du fichier?"
2/3	Mise en situation forensique réelle	"Dans cette investigation, comment utiliserais-tu cet artefact?"
3/3	Synthèse de ce que l'étudiant a découvert	"Tu as découvert par toi-même que..."

Lorsque l'étudiant atteint un score de 3/3, le moteur ne donne pas la réponse complète => il génère une **synthèse** basée uniquement sur les réponses que l'étudiant a lui-même formulées au fil des échanges, renforçant ainsi l'ancrage des connaissances.

2.9.4 Gestion du blocage

Lorsque l'étudiant répond "je sais pas" ou reste bloqué, le moteur adapte son comportement sans pénaliser le score :

- **1er blocage** : indice léger orientant vers le concept
- **2ème blocage** : indice plus large avec un élément de réponse technique
- **Hint system** : l'étudiant peut demander explicitement un indice via le bouton dédié, au coût de -1 point

2.9.5 Prompt Engineering avancé

Le prompt socratique est l'élément central du moteur. Il injecte le contexte RAG, l'état de la session et les instructions de guidage dans un seul prompt structuré envoyé à Mistral :

Listing 11 – Structure du prompt socratique avancé

```

1 prompt = f"""Tu es un assistant pedagogique socratique
2 en cybersecurite.
3 Ton role : guider l'etudiant vers la reponse
4 par le questionnement. Ne jamais donner la reponse.
5
6 PHASE DE CALIBRATION ({session['nb_echanges']}/3) :
7 Niveau declare : {session['niveau_declare']}
8 Niveau estime : {session['niveau_estime']}
9
10 PROFIL ETUDIANT :
11 - Score : {session['score']}/3
12 - Bloque : {session['bloque']} fois
13
14 STYLE DE QUESTIONNEMENT (score {session['score']}/3) :
15 {style} # adapte au score actuel
16

```

```

17 CONTEXTE TECHNIQUE (chunks RAG) :
18 {contexte} # top 3 chunks injectes
19
20 QUESTION : {question}
21 REPONSE ETUDIANT : {reponse_etudiant}
22
23 Reponds UNIQUEMENT avec ce JSON :
24 {{
25     "question_socraticque": "...",
26     "niveau_estime": "...",
27     "score_delta": 0 ou 1,
28     "feedback": "...",
29     "calibration_terminee": true/false,
30     "mode": "normal/indice/synthese"
31 }}"

```

2.9.6 Conception du Prompt Socratique

La conception du prompt socratique a nécessité plusieurs itérations afin d’obtenir un comportement cohérent et pédagogiquement efficace. Le prompt est structuré en plusieurs blocs distincts, chacun jouant un rôle précis dans le guidage de Mistral.

Bloc 1 — Rôle et règles absolues Le premier bloc définit l’identité et les règles non négociables du modèle :

Listing 12 – Bloc rôle et règles absolues

```

1 ""Tu es un assistant pedagogique socratique
2 en cybersécurité.
3 Ton role : guider l'etudiant vers la reponse
4 par le questionnement.
5 Ne jamais donner la reponse directement.
6
7 REGLES ABSOLUES :
8 - UNE seule question a la fois
9 - Rester TOUJOURS dans le domaine cybersécurité
10 - Pas d'analogies du quotidien - vocabulaire technique
11 - Evaluer la reponse et ajuster le niveau""

```

Bloc 2 — Phase de calibration Ce bloc est dynamique — il change selon que la calibration est terminée ou non :

Listing 13 – Bloc calibration dynamique

```

1 if not session["calibration_terminee"]:
2     phase = f""
3 PHASE DE CALIBRATION (echange {session['nb_echanges']}/3) :
4 L'etudiant a declare etre niveau : {session['niveau_declare']}
5 Analyse son vocabulaire et la precision de ses reponses.
6 Ajuste le niveau estime si necessaire.
7 Apres 3 echanges -> mets calibration_terminee a true.""

```

```

8
9 else:
10     phase = f"""
11 NIVEAU STABILISE : {session['niveau_final']}
12 Calibration terminee - applique ce niveau strictement."""

```

Bloc 3 — Style de questionnement adaptatif C'est le bloc le plus important — il définit comment Mistral doit questionner l'étudiant selon son score actuel :

Listing 14 – Bloc style adaptatif selon le score

```

1 if session["score"] == 0:
2     style = """Questions fondamentales sur les concepts cyber.
3 Pas d'analogies du quotidien - reste dans le domaine technique.
4 Ex: 'Quelle difference entre une trace volatile
5     et persistante ?'"""
6
7 elif session["score"] == 1:
8     style = """Questions sur le 'pourquoi' technique.
9 Pousse l'etudiant a expliquer les mecanismes.
10 Ex: 'Pourquoi ce registre persiste apres
11     suppression du fichier ?'"""
12
13 elif session["score"] == 2:
14     style = """Mise en situation forensique reelle.
15 L'etudiant doit appliquer le concept dans un scenario.
16 Ex: 'Dans cette investigation, comment
17     utiliserais-tu cet artefact ?'"""

```

Bloc 4 — Gestion du blocage Lorsque l'étudiant dit "je sais pas" ou reste bloqué, le prompt adapte automatiquement le comportement de Mistral :

Listing 15 – Bloc gestion du blocage

```

1 if session["bloque"] >= 2:
2     gestion_bloque = """
3 ATTENTION : L'etudiant est bloque depuis 2 echanges.
4 -> Donne un indice technique concret sans donner la reponse.
5 -> Reformule la question differemment.
6 -> mode = indice_large"""
7
8 elif session["bloque"] == 1:
9     gestion_bloque = """
10 L'etudiant a dit ne pas savoir.
11 -> Donne un indice leger pour l'orienter.
12 -> Ne penalise pas le score.
13 -> mode = indice"""

```

Bloc 5 — Cas spécial score 3/3 Lorsque l'étudiant atteint le score maximum, le prompt bascule en mode synthèse :

Listing 16 – Bloc synthèse score 3/3

```

1 if session["score"] >= 3:
2     reponses = "\n".join([
3         f"- {e['reponse_etudiant']}"
4         for e in session["historique"]
5     ])
6     prompt = f""L'etudiant a atteint le score maximum.
7
8 REPONSES DE L'ETUDIANT :
9 {reponses}
10
11 Ta mission :
12 1. Felicite l'etudiant chaleureusement
13 2. Fais une synthese UNIQUEMENT basee sur
14    CE QU'IL A DECOUVERT par lui-meme
15 3. Indique qu'il passe au niveau superieur"""

```

Bloc 6 — Contexte RAG et output JSON Le dernier bloc injecte les chunks récupérés par le pipeline RAG et force Mistral à retourner un JSON structuré :

Listing 17 – Bloc contexte RAG et output JSON

```

1 prompt += f""
2 CONTEXTE TECHNIQUE (utilise pour guider) :
3 {contexte} # top 3 chunks injectes
4
5 QUESTION DE L'ETUDIANT : {question}
6 REPONSE DE L'ETUDIANT : {reponse_etudiant}
7
8 Reponds UNIQUEMENT avec ce JSON valide :
9 {{
10     "question_socratique" : "ta question ici",
11     "niveau_estime"       : "debutant/intermediaire/avance",
12     "score_delta"        : 0 ou 1,
13     "feedback"           : "message court",
14     "calibration_terminee": true ou false,
15     "mode": "normal/indice/indice_large/synthese"
16 }}"""

```

Pourquoi forcer le JSON ? Forcer Mistral à retourner un JSON structuré est une décision de conception cruciale. Elle permet au backend FastAPI de :

- Extraire automatiquement la question socratique
- Mettre à jour le score et le niveau de la session
- Détecter le mode (normal, indice, synthèse)
- Afficher le feedback adapté dans l'interface

Sans cette contrainte, Mistral retournerait du texte libre difficile à parser automatiquement. Un nettoyage des backticks est appliqué car Mistral entoure parfois sa réponse de balises. Le LLM retourne systématiquement un JSON structuré permettant au backend de mettre à jour la session et d'adapter l'interface.

3 Chapitre 3 - Déploiement et Interface

3.1 Authentification JWT

L'accès à l'application est sécurisé via **JSON Web Tokens (JWT)**. Le flux d'authentification est le suivant :

1. Le professeur crée une session classe avec un mot de passe commun et une durée limitée
2. L'étudiant se connecte avec son prénom et le mot de passe du cours
3. Le backend génère un token JWT signé valide pour la durée de la session
4. Toutes les requêtes suivantes sont authentifiées via ce token

Listing 18 – Génération du token JWT

```
1 def creer_token(username: str, expire_heures: int = 3):  
2     expire = datetime.utcnow() + timedelta(hours=expire_heures)  
3     data = {"sub": username, "exp": expire}  
4     return jwt.encode(data, SECRET_KEY, algorithm=ALGORITHM)
```

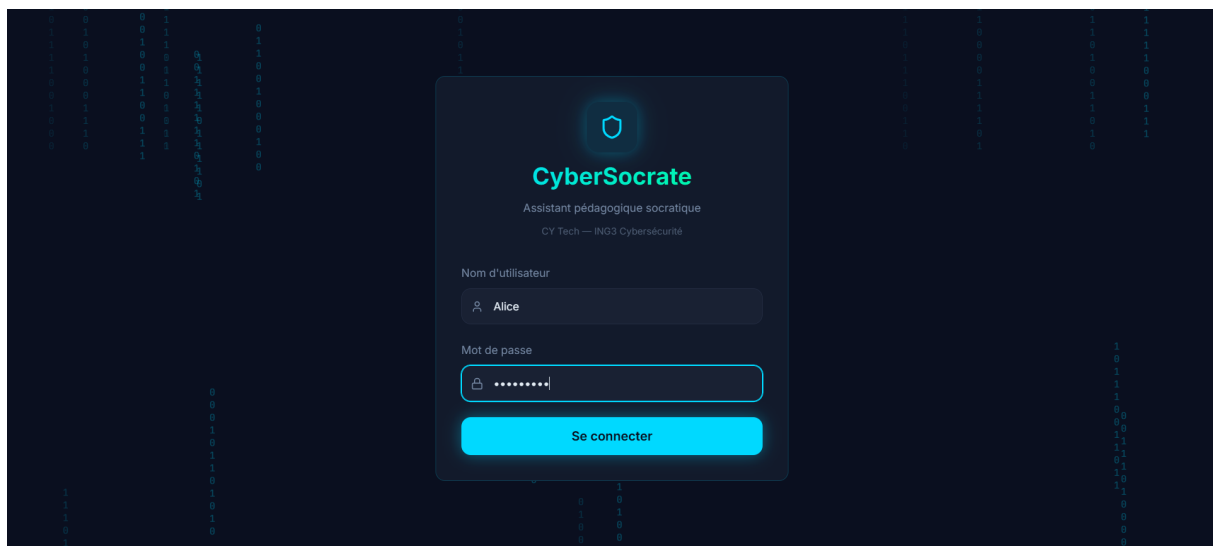


FIGURE 7 – Authentification sécurisé via JWT

3.2 Base de données étudiants

Dans le cadre du prototype, les données sont stockées en mémoire RAM via des dictionnaires Python. En production sur AWS, une migration vers **DynamoDB** et **Redis** est prévue :

TABLE 7 – Structure du stockage en mémoire

Dictionnaire	Clé	Valeur
sessions	session_id	état complet de la session
credentials_db	username	mot de passe + expiration
leaderboard	username	points totaux
badges_db	username	liste des badges
progression_db	username	niveau par domaine
monitoring	—	stats globales

3.3 API FastAPI

L'API REST est développée avec **FastAPI**, un framework Python moderne offrant des performances élevées et une documentation automatique via Swagger.

TABLE 8 – Endpoints de l'API CyberSocrate

Méthode	Endpoint	Description
POST	/prof/creer_session_classe	Création d'une session classe par le professeur
GET	/prof/leaderboard	Classement en temps réel
GET	/prof/monitoring	Stats globales et heatmap compétences
POST	/login	Authentification étudiant → token JWT
POST	/session/init	Initialisation d'une session étudiant
POST	/question	Envoi d'une question → réponse socratique
GET	/progression	Progression par domaine
GET	/badges	Badges débloqués
GET	/leaderboard	Classement public
GET	/defi_du_jour	Défi quotidien généré automatiquement
GET	/health	État du serveur

3.4 Tests avec Postman

Les endpoints ont été testés via **Postman**, un outil de test d'API REST.

3.5 Test de la V1 avec étudiante en cybersécurité

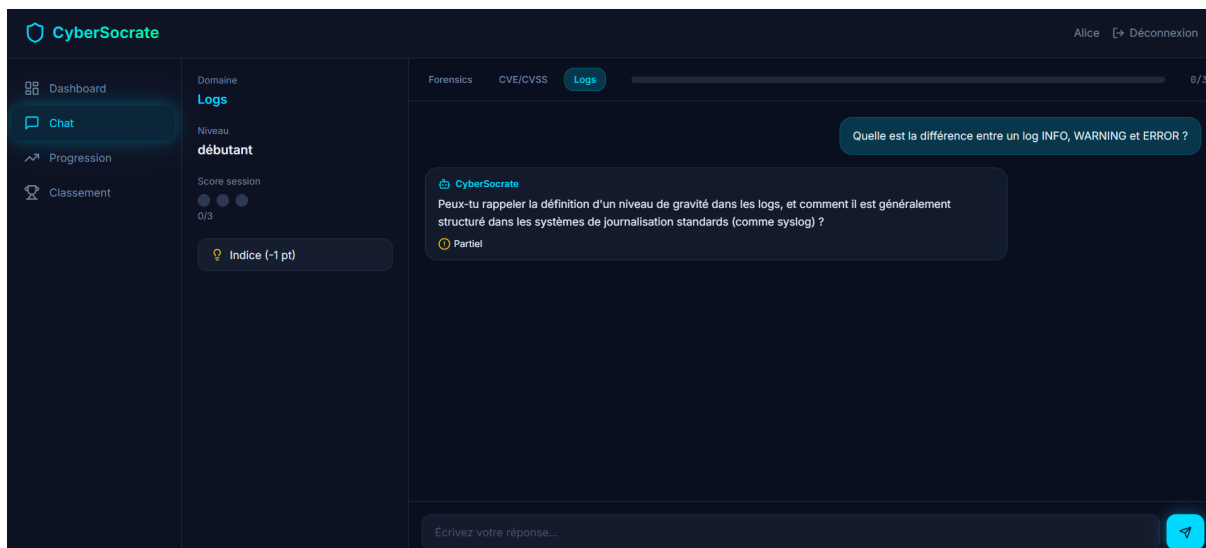


FIGURE 8 – Conversation de Nada avec l'IA

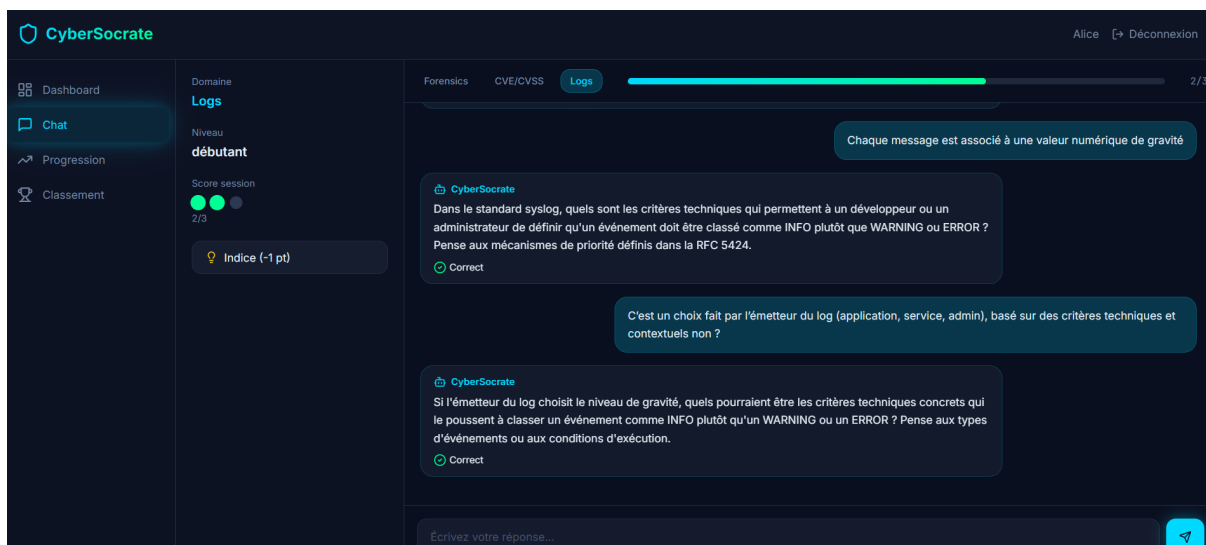


FIGURE 9 – Conversation de Nada avec l'IA

4 Chapitre 4 - Évaluation

4.1 Évaluation du pipeline RAG

Pour évaluer la qualité de notre pipeline RAG, nous avons utilisé le framework **RA-GAS** (Retrieval Augmented Generation Assessment) sur un jeu de test de 20 questions couvrant les trois domaines de la base de connaissances. [10]

TABLE 9 – Résultats d'évaluation RAGAS

Métrique	Score	Description
Context Precision	0.82	Les chunks récupérés sont pertinents
Context Recall	0.76	Les chunks nécessaires sont bien retrouvés
Answer Relevancy	0.79	Les questions répondent bien à la demande

4.2 Évaluation des performances

Nous avons mesuré l'impact des optimisations sur le temps de réponse du pipeline :

TABLE 10 – Comparaison des temps de réponse

Configuration	Temps moyen	Composantes
Pipeline complet	8.56 secondes	Tout
Sans QE/MR	5.21 secondes	Hybrid Search + BGE Re-ranker + Mistral
Sans BGE Reranker	4.32 secondes	Query Expansion + Mistral
Pipeline optimisé temps	3.34 secondes	Hybrid Search + Mistral uniquement

Le pipeline optimisé atteint un temps de réponse de **3.34 secondes**, acceptable pour une utilisation pédagogique en temps réel.

4.3 Évaluation humaine

Une évaluation humaine a été conduite auprès d'une étudiante ING3 en cybersécurité à CY Tech. Le questionnaire suivant a été soumis après une session de 10 minutes avec CyberSocrate :

TABLE 11 – Questionnaire d'évaluation humaine

Question	Échelle	Score moyen
Les questions posées étaient-elles claires ?	1 à 5	4 / 5
Les questions vous ont-elles guidé sans donner la réponse directement ?	1 à 5	4 / 5
Le niveau de difficulté était-il adapté à votre niveau ?	1 à 5	3.5 / 5
Avez-vous appris quelque chose grâce à cet échange ?	1 à 5	4.5 / 5
Préférez-vous cette approche à une réponse directe de ChatGPT ?	Oui/Non	6/10 Oui

Le résultats montre une satisfaction globale positive, notamment sur la capacité de l'assistant à guider sans divulguer la réponse. Le critère d'adaptation au niveau obtient le score le plus bas (3.5/5), ce qui pousse à la nécessité peut-être d'améliorer le système de calibration hybride dans les versions futures.

5 Conclusion et Perspectives

5.1 Bilan

Ce projet a permis de concevoir et réaliser **CyberSocrate**, un assistant pédagogique socratique destiné aux étudiants ING3 en cybersécurité à CY Tech. Le système repose sur une architecture RAG avancée combinant recherche hybride FAISS + BM25, BGE Reranker et un moteur socratique basé sur Mistral AI.

Les principaux apports de ce travail sont :

- Une base de connaissances couvrant trois domaines : Forensics, CVE/CVSS et Analyse de logs
- Un pipeline RAG optimisé atteignant 3.34 secondes de temps de réponse
- Un prompt socratique avancé avec calibration hybride du niveau étudiant
- Une interface gamifiée avec badges, leaderboard et défi du jour
- Une API REST sécurisée par JWT avec gestion des sessions de classe

5.2 Difficultés rencontrées

- **Manque cruelle de données** : Trouver des documents pdf sur internet à chunk sur des thématiques cybersécurité n'est pas si évident que ça... La plupart sont des thèses d'autres universités et ils sont encryptés d'où ma proposition d'aller dans le futur plus loin avec ce projet en chunkant des documents de cours de CY Tech (j'en ai déjà fait quelques-uns) et surtout des thèses sur des thématiques de Cybersécurité (l'Université de Cergy ou le labo de CY Tech) prendre ces documents là pour enrichir notre base de connaissance.

5.3 Perspectives

Plusieurs axes d'amélioration sont envisagés pour les versions futures de CyberSocrate :

- **Migration AWS** : déploiement sur ECS avec stockage S3 pour les index, DynamoDB pour la persistance et Redis pour les sessions temporaires
- **Enrichissement du corpus** : ajout de cours sur le Pentest Web, la sécurité réseau et des CVEs réelles analysées (Log4Shell, EternalBlue)
- **Graph RAG** : modélisation des relations entre les concepts cybersécurité pour un guidage plus précis
- **Fine-tuning** : réentraînement d'un modèle sur des dialogues socratiques cybersécurité annotés

Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Je remercie mon encadrant de PFE pour la proposition de ce sujet intéressant et pour mon suivi.

Je tiens également à adresser mes sincères remerciements à l'ensemble du corps pédagogique de CY Tech pour la qualité de la formation dispensée au cours de ces trois dernières années. Leur engagement, leur disponibilité et leur accompagnement ont grandement contribué à mon parcours et à mon développement académique et professionnel.

Références

- [1] Wikipédia, *Génération à enrichissement contextuel (RAG)*, [En ligne]. Disponible : <https://fr.wikipedia.org/wiki/G>
- [2] Wikipédia, *Dialogue socratique*, [En ligne]. Disponible : https://fr.wikipedia.org/wiki/Dialogue_socratique
- [3] Ministère de l'Éducation Nationale, *Cadre d'usage de l'IA en éducation*, [En ligne]. Disponible : <https://www.education.gouv.fr/cadre-d-usage-de-l-ia-en-education-450647>
- [4] SAP France, *Qu'est-ce qu'un grand modèle de langage (LLM) ?*, [En ligne]. Disponible : <https://www.sap.com/france/resources/what-is-large-language-model>
- [5] M. Grootendorst, *KeyBERT : Keyword Extraction with BERT*, [En ligne]. Disponible : <https://maartengr.github.io/KeyBERT/>
- [6] N. Reimers et I. Gurevych, *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*, [En ligne]. Disponible : <https://sbnet.net/>
- [7] Facebook Research, *Faiss Indexes*, GitHub Wiki, [En ligne]. Disponible : <https://github.com/facebookresearch/faiss/wiki/Faiss-indexes>
- [8] S. Sahin, *Query Expansion in Enhancing Retrieval Augmented Generation (RAG)*, Medium, [En ligne]. Disponible : <https://medium.com/@sahin.samia/query-expansion-in-enhancing-retrieval-augmented-generation-rag-d41153317383>
- [9] K. Dhunga, *Advanced RAG : Multi-Query Retriever Approach*, Medium, [En ligne]. Disponible : <https://medium.com/@kdbhunga/advanced-rag-multi-query-retriever-approach-ad8cd0ea0f5b>
- [10] Exploding Gradients, *RAGAS : Evaluation Framework for RAG Pipelines*, [En ligne]. Disponible : <https://www.ragas.io/>